Lab No: 07

❋ **Lab Title: Arrays ○ Learning objectives:**

The following topics will be covered in this lab session

- What is an Array
- When to use Array
- Declaration of array
- Initialization of Array
- Accessing Array element
- Copying arrays Character Array (String)

What is Array?

Array is a data structure in C++, which stores a fixed size sequential collection of elements of the same type. An array is used to store a collection of data. It is often useful to think of an array as a collection of variables of the same type.

Instead of declaring individual variables, such as number0, number1, ..., and number99, you declare one array variable such as numbers and use numbers [0], numbers [1], and ..., numbers [99] to represent individual variables. A specific element in an array is accessed by an index.

All arrays consist of contiguous memory locations. The lowest address corresponds to the first element and the highest address to the last element.

↳ When to use Array:

Let's start by looking at listing 1 where a single variable is used to store a person's age.

```
#include  <iostream>
using namespace std;
int main ()
{
        int age;
        age=21;
        cout<<age<<endl;
        return 0;
}
```

Now let's keep track of 4 ages instead of just one. We could create 4 separate variables, but creating 4 separate variables is not a good approach. (If you are using 4 separate variables for it, then consider keeping track of 1000 ages instead of just 4). Rather than using 4 separate variables, we'll use an array because it is quite easy to handle one variable instead of 1000.

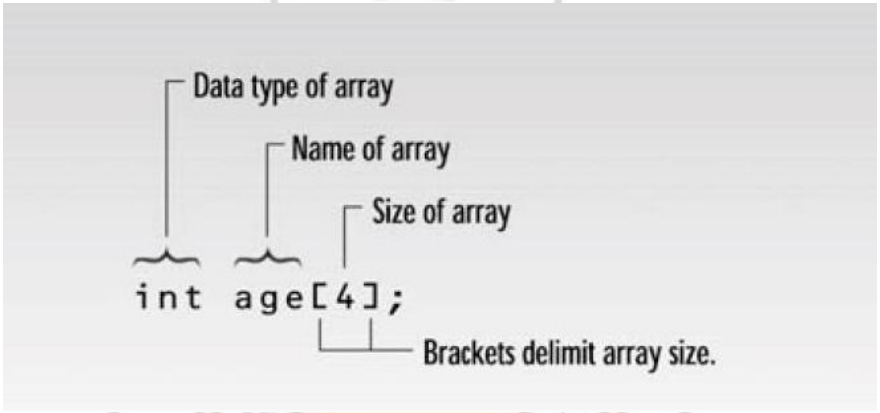
Declaration of Array:

To declare an array in C++, the programmer specifies the type of the elements and the number of elements required by an array as follows:

type arrayName [*arraySize*];

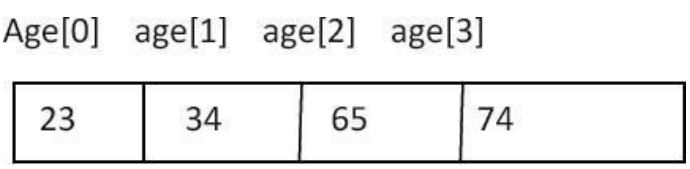
This is called a single-dimension array. The **array Size** must be an integer constant greater than zero and **type** can be any valid C++ data type. For example, to declare a 4-element array called age of type int, use this statement:

int age [4];




```
#include <iostream>
using namespace std;
int main()
{
        int age[4];          //declaration of Array
                age[0]=23;          //initialization of Array elements
                age[1]=34;
                age[2]=65;
                age[3]=74;
 return 0;
}
```

On line (5), an array of 4 int is created. Values are assigned to each variable in the array on line (6) through line (9). In memory these are contiguous set of locations shown in following figure.




```
#include <iostream>
using namespace std;
int main()
{
        int age[4];     //array 'age' of 4 ints
                for(int j=0; j<4; j++)          //get 4 ages
                {
                        cout << "Enter an age: ";
                        cin >> age[j];   //access array element
                }
        for(j=0; j<4; j++) //display 4 ages
                cout << "You entered " << age[j] << endl;
        return 0;
}
```

 Here's a sample interaction with the program in example.

```
Enter an age: 44
Enter an age: 16
Enter an age: 23
Enter an age: 68

You entered 44

You entered 16
You entered 23
You entered 68
```

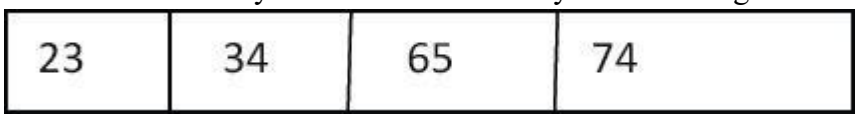
↳ Initialization of Array:

It is like a variable; an array can be initialized. To initialize an array, we provide initializing values which are enclosed within curly braces in the declaration and placed following an equals sign after the array name. Here is an example of initializing an integer array.

int age[4]={23,34,65,74};

age[0] age[1] age[2] age[3]

Now how to initialize all the values in array to 0? It can be done by the following statement;



int age[4]={0}; age[0]

age[1] age[2] age[3]

↳ Accessing elements of Array:

In any point of a program in which an array is visible, we can access the value of any of its elements individually as if it was a normal variable, thus being able to both read and modify its value. array name[index];

Following the previous examples in which "age" had 4 elements and each of those elements was of type int, the name which we can use to refer to each element is the following:

age [0] age [1] age [2] age [3]

For example, to store the value 75 in the third element of age , we could write the following statement: age[2]=75; //**note** : array index start with 0 in c.

And, for example, to store the value of the third element of age to a variable called a, we could write:

int a=age [2];

Here's another example of an array at work. This one, SALES, invites the user to enter a series of six values representing widget sales for each day of the week (excluding Sunday), and then calculates the average of these values. We use an array of type double so that monetary values can be entered.

```
#include <iostream>
using namespace std;
int main()
{
        const int SIZE = 6;             //size of array
        double sales[SIZE];             //array of 6 variables
        cout << "Enter widget sales for 6 days\n";
                for(int j=0; j<SIZE; j++)       //put figures in array
                        cin >> sales[j];
        double total = 0;
                for(j=0; j<SIZE; j++)           //read figures from array
                        total += sales[j]; //to find total
        double average = total / SIZE; // find average
        cout << "Average = " << average << endl;
        return 0;
}
```

Here's some sample interaction with SALES:

 Enter widget sales for 6 days:

352.64

867.70

781.32

867.35

746.21

189.45

Average = 634.11

↳ Copying arrays:

Suppose that after filling our 4-element array with values, we need to copy that array to another array of 4 int? Try this:

```
#include <iostream>
    using namespace std;
int main()
{
                int age[4];
                int same_age[4];
                int i=0;
                                age[0]=23;
                                age[1]=34;
                                age[2]=65;
                                age[3]=74;
                for (;i<4;i++)
                                same_age[i]=age[i];
                for (i=0;i<4;i++)
                                cout<<same_age[i];
return 0;
}
```

In the above program two arrays are created: age and same age. Each element of the age array is assigned a value. Then, in order to copy the four elements in age into the same age array, we must do it element by element. We have used for loop to access every element of array note that for loop takes value from 0 to
3.

Note: Like printing arrays, there is no single statement in the language that says "copy an entire array into another array". The array elements must be copied individually. Thus If we want to perform any action on an array, we must repeatedly perform that action on each element in the array.

↳ Exercises:

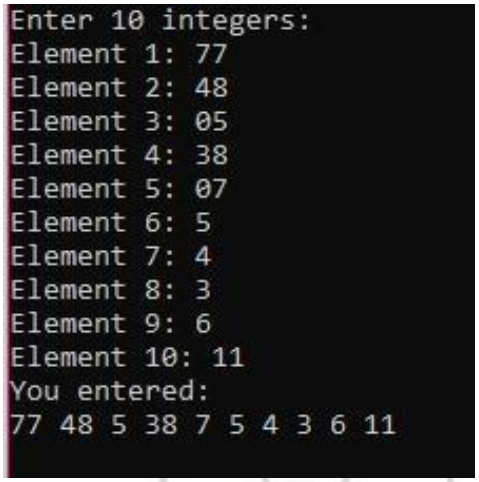
Q1. Declare an integer array of size 10. Get all the values of the array elements from the user (using loop). Use another loop to display all these elements.

□ Input:

```
#include <iostream> usingnamespace
std;
int main () {
   int arr[10];
   cout << "Enter 10 integers:" << endl;
   for(int i = 0; i < 10; i++) {
      cout << "Element " << i + 1 << ": ";
      cin >> arr[i]; }
   cout<< "You entered:"<< endl; for(int
   i = 0; i < 10; i++) {
```

```
    cout << arr[i] << " "; }
  cout << endl;
  return 0;}
```

- **Output:**



Q2. Declare an array of size 8. Get all the elements from the user using loop. Display the entered elements in reveres order.

- **Input:**

```
#include  <iostream>
using namespace std;
int main() {
   int arr[8];
   cout << "Enter 8 integers:" << endl;
   for(int i = 0; i < 8; i++) {
      cout << "Element " << i + 1 << ": ";
      cin >> arr[i];
   }
   cout << "You entered (in reverse order):" << endl;
   for(int i = 7; i >= 0; i--) {
      cout << arr[i] << " ";
   }
   cout << endl;
 return 0;
}
```

Output:

```
Enter 8 integers:
Element 1: 3
Element 2: 4
Element 3: 5
Element 4: 6
Element 5: 7
Element 6: 8
Element 7: 9
Element 8: 1
You entered (in reverse order):
1 9 8 7 6 5 4 3
```